

Unidade II

3 ABSTRAÇÃO E INTERFACES

Em desenvolvimento de software orientado a objetos, muitos problemas cotidianos de manutenção têm origem em decisões estruturais tomadas ainda nas primeiras versões do sistema. Sem um uso consciente de abstração, encapsulamento e contratos, o código tende a acumular dependências implícitas, trechos duplicados e suposições frágeis sobre o comportamento das classes.

O resultado é um cenário em que alterar uma funcionalidade aparentemente simples exige percorrer uma cadeia de tipos relacionados, com o risco constante de introduzir comportamentos inesperados.

A discussão sobre como organizar hierarquias, ocultar detalhes internos e definir pontos claros de extensão torna-se, portanto, uma preocupação central para quem deseja que o software continue evoluindo com segurança.

Dentro desse contexto, C# oferece mecanismos específicos para estruturar essas decisões: classes abstratas e membros abstratos permitem modelar conceitos gerais que não fazem sentido, como objetos concretos, concentrando ali o código compartilhado e delegando às subclasses a responsabilidade por especializações obrigatórias.

Ao mesmo tempo, o uso adequado de modificadores de acesso, construtores em classes base e fatoração de lógica comum favorece a ocultação de detalhes de implementação e a exposição de interfaces mais enxutas ao código cliente.

Este tópico trabalha tais elementos com exemplos didáticos, como a hierarquia de formas geométricas, para mostrar como uma boa escolha de classe base e de contrato abstrato conduz a um modelo mais claro e reutilizável em C#.

As interfaces aparecem como outro pilar dessa arquitetura, agora enfatizando contratos puros e independentes de qualquer hierarquia concreta. Ao definir apenas assinaturas de métodos, propriedades e outros membros, as interfaces permitem que tipos de linhagens distintas assumam capacidades comuns e sejam tratados de forma uniforme por algoritmos polimórficos.

A linguagem ainda acrescenta a possibilidade de implementação explícita de interface, oferecendo um controle mais refinado sobre quais membros ficam disponíveis pela visão pública da classe e como resolver conflitos de nome quando múltiplas interfaces exigem operações com a mesma assinatura.

Esses recursos respondem a necessidades reais de design, em que uma classe participa de vários papéis e precisa separar cuidadosamente suas diferentes faces contratuais.

Por fim, a discussão ganha profundidade ao relacionar esse aparato técnico ao princípio da substituição de Liskov (LSP), que funciona como critério para julgar se uma hierarquia ou uma implementação de interface é saudável.



Saiba mais

O livro a seguir é um catálogo clássico dos 23 padrões de projeto do Gang of Four (GoF), cobrindo padrões de criação, estrutura e comportamento no contexto da engenharia de software. A obra é útil para estudar o LSP e demais princípios de bom design porque mostra soluções estruturadas que reforçam encapsulamento, baixo acoplamento e alta coesão – justamente o tipo de contexto em que o SOLID faz mais sentido.

GAMMA, E. *et al. Padrões de projeto: soluções reutilizáveis de software orientado a objetos*. Porto Alegre: Bookman, 2005.

Já no livro seguinte sistematizam-se princípios de design orientado a objetos, entre eles o conjunto que ficou conhecido como SOLID.

MARTIN, R. C.; MARTIN, M. *Princípios, padrões e práticas ágeis em C#*. Porto Alegre: Bookman, 2011.

O exame de exemplos clássicos, como o dilema entre quadrado e retângulo, mostra que não basta herdar ou implementar para que a substituição funcione de forma segura: o tipo derivado precisa preservar as expectativas lógicas do tipo base, sem introduzir restrições ocultas nem comportamentos surpreendentes para o código cliente.

Ao articular classes abstratas, interfaces, implementação explícita e LSP, este tópico prepara o leitor para enxergar esses recursos de C# não como detalhes de sintaxe isolados, mas como instrumentos de projeto que, se bem utilizados, contribuem para sistemas mais claros, estáveis e evolutivos.

3.1 Classes e membros abstratos, ocultação de detalhes e contratos

Elaborar sistemas orientados a objetos de qualidade envolve compreender princípios fundamentais como abstração, encapsulamento e contratos. No contexto da programação em C#, a abstração se manifesta por meio de classes abstratas e membros abstratos, enquanto o encapsulamento diz respeito à ocultação de detalhes de implementação.

Além disso, o conceito de contrato aparece principalmente nas interfaces, que definem conjuntos de funcionalidades que as classes devem prover. Esses elementos trabalham em conjunto para produzir códigos mais modularizados, flexíveis e de fácil manutenção, ao separar nitidamente o "o quê" do "como" em uma aplicação.

A abstração, em ciência da computação, consiste em focar os aspectos essenciais de um objeto ou um sistema, omitindo detalhes supérfluos. Isso permite modelar conceitos do mundo real de forma simplificada, apropriada ao problema que se deseja resolver.

Por exemplo, ao abstrair um objeto "forma geométrica", interessam-nos propriedades e comportamentos gerais como cálculo de área, mas não detalhes específicos de cada forma concreta. Em C#, a abstração é suportada diretamente pelas classes abstratas. De acordo com Deitel e Deitel (2016), **uma classe abstrata é aquela que não pode ser instanciada diretamente**, ou seja, não é possível criar objetos concretos dela.

Ela existe principalmente para servir de modelo ou classe base a outras classes mais especializadas. Uma classe abstrata frequentemente contém definições de métodos abstratos, os quais também não possuem implementação e devem obrigatoriamente ser substituídos (`override`) pelas subclasses concretas.

Assim, ao declarar um método como abstrato, a classe base está adiando sua implementação para as subclasses, garantindo que cada subclasse forneça sua versão específica do comportamento requerido. Esse mecanismo força um contrato entre a superclasse e as subclasses: quem herda de uma classe abstrata se compromete a implementar todos os membros abstratos herdados, assegurando que certos métodos existam na classe derivada.

Caso a subclasse não implemente algum método abstrato herdado, ela própria passa a ser considerada abstrata pelo compilador (devendo, portanto, também ser declarada com o modificador `abstract`).

Uma classe abstrata pode definir tanto membros abstratos quanto membros concretos (não abstratos). Membros abstratos (que incluem métodos, propriedades, indexadores ou eventos) são declarados com o modificador `abstract` e não possuem corpo; já os membros não abstratos possuem implementação normal.

É permitido que uma classe abstrata não contenha nenhum método abstrato; nesse caso, marcá-la como abstrata serve apenas para impedir sua instanciação direta e possivelmente para indicar que ela é incompleta conceitualmente.

Classes abstratas podem inclusive definir campos e construtores. Embora não se possa criar instâncias de uma classe abstrata, suas subclasses podem invocar o construtor base (da classe abstrata) para reutilizar lógica de inicialização comum. Assim, classes abstratas possibilitam **fatorar** código compartilhado e ocultar detalhes de implementação comuns às subclasses, ao mesmo tempo que definem um conjunto de funcionalidades que as classes derivadas devem personalizar.



Observação

Fatorar significa reorganizar o código de modo a extrair o que é comum a várias classes e concentrar isso em um único ponto, em vez de repetir a mesma lógica em cada classe concreta.

Em orientação a objetos, isso costuma envolver identificar atributos, construtores e métodos que aparecem com a mesma função em várias subclasses e movê-los para uma superclasse – aqui, uma classe abstrata.

Quando afirmamos que classes abstratas permitem fatorar código compartilhado, a ideia é justamente usar a classe abstrata como local central no qual ficam definidos os dados e a lógica de inicialização que todas as subclasses têm em comum. As subclasses, ao chamar o construtor da classe abstrata, reaproveitam essa lógica já implementada, garantindo que a parte comum do processo seja executada sempre da mesma maneira. Ao mesmo tempo, cada subclasse implementa ou sobrescreve os detalhes específicos que a distinguem das demais.

Para ilustrar, imagine uma pequena hierarquia de formas geométricas. Poderíamos modelar uma classe abstrata `Forma` que define um método abstrato `CalcularArea()` – pois cada forma geométrica concreta tem sua própria fórmula de área – e também fornece alguma funcionalidade comum, como um nome ou uma descrição genérica.

```
// Classe abstrata base
public abstract class Forma
{
    public string Nome { get; protected set; }
    // Construtor da classe abstrata base
    protected Forma(string nome) {
        Nome = nome;
    }

    // Membro abstrato: nenhuma implementação aqui
    public abstract double CalcularArea();
    // Método concreto: implementado na classe abstrata
    public void Descrever()
    {
        Console.WriteLine($"Esta forma é um {Nome}.");
    }
}

// Subclasse concreta da forma
public class Circulo : Forma
{
    private double raio;
```

```
public Circulo(double raio) : base("círculo") {
    this.raio = raio;
}

// Implementação obrigatória do método abstrato
public override double CalcularArea()
{
    return Math.PI * raio * raio;
}

// Outra subclasse concreta
public class Retangulo : Forma
{
    private double largura, altura;

    public Retangulo(double largura, double altura) : base("retângulo") {
        this.largura = largura;
        this.altura = altura;
    }

    public override double CalcularArea()
    {
        return largura * altura;
    }
}
```

Nesse código, **Forma** é uma classe abstrata que define o contrato de que toda forma geométrica deve saber calcular sua área (**CalcularArea**). As subclasses **Circulo** e **Retangulo** herdam de **Forma** e implementam o método abstrato com as fórmulas específicas para área de círculo e retângulo, respectivamente. Observe que **Forma** provê também um método concreto **Descrever()**, que imprime uma descrição genérica usando um atributo comum (Nome).

Essa implementação compartilhada é herdada pelas subclasses tal como está, por exemplo, tanto um **Circulo** quanto um **Retangulo** podem usar **Descrever()** diretamente, sem precisar reescrever esse código. Assim, a classe base oculta os detalhes de como a descrição é produzida; as subclasses simplesmente reutilizam essa funcionalidade.

Note que não podemos instanciar **Forma** diretamente. Qualquer tentativa de executar **new Forma** ("alguma") resultaria em erro de compilação, pois **Forma** é abstrata. Em vez disso, instanciamos as subclasses concretas. Contudo, graças ao polimorfismo, podemos tratar os objetos derivados como objetos do tipo abstrato. Por exemplo:

```
Forma f1 = new Circulo(2.5);
Forma f2 = new Retangulo(3.0, 4.0);

Console.WriteLine( f1.CalcularArea() );    // Chama Circulo.CalcularArea()
Console.WriteLine( f2.CalcularArea() );    // Chama Retangulo.CalcularArea()
f1.Descrever();    // Usa Forma.Descrever() para um círculo
f2.Descrever();    // Usa Forma.Descrever() para um retângulo
```

Nesse trecho, as variáveis `f1` e `f2` são do tipo abstrato **Forma**, mas referenciam instâncias concretas de **Circulo** e **Retangulo**. Ao invocar `CalcularArea()` em `f1` e `f2`, o método apropriado de cada subclasse é executado (graças ao **override** polimórfico). Isso demonstra um benefício-chave das classes abstratas: elas permitem o programar orientado à abstração, manipulando objetos diferentes através de uma interface comum.

O código que usa **Forma** não precisa conhecer os detalhes internos de cada subclasse para chamar `CalcularArea()`, ele sabe apenas que toda **Forma** tem esse método, conforme definido no contrato da classe base. Os detalhes do cálculo ($\pi \times r^2$ para círculo, base x altura para retângulo etc.) estão ocultos dentro das respectivas classes concretas.

Essa ocultação de detalhes de implementação reduz a complexidade e aumenta a flexibilidade do software: podemos adicionar novas formas (ou modificar as existentes) sem alterar o código que trabalha com o tipo abstrato **Forma**, desde que o contrato (a interface pública) seja mantido.



Lembrete

O ato de ocultar detalhes de implementação está intimamente ligado ao princípio do encapsulamento que vimos anteriormente. Encapsular significa agrupar dados e métodos que operam sobre esses dados em uma mesma unidade (a classe) e controlar o acesso aos detalhes internos desse objeto.

A classe expõe apenas o que for necessário através de membros públicos (sua interface pública) e restringe o acesso às partes internas, marcando-as como privadas ou protegidas. No exemplo, os atributos `raio`, `largura` e `altura` nas subclasses são privados, logo, o código externo não pode modificá-los diretamente, garantindo integridade. Se for preciso ler ou modificar esses valores, a classe poderia fornecer métodos ou propriedades públicas controladas. Esse controle é a essência da ocultação de dados: os usuários da classe interagem com ela através de métodos e propriedades públicos, sem precisar (nem poder) saber como ela armazena seus dados ou implementa suas funcionalidades internamente.

A vantagem disso é dupla: por um lado, evita que outras partes do programa dependam de detalhes específicos (que podem mudar) e, por outro, proporciona uma interface clara de uso. Combinando encapsulamento com abstração, obtemos um design em que cada classe é uma caixa-preta bem definida: sabemos o que ela faz, mas não necessariamente como o faz internamente.

Enquanto as classes abstratas fornecem uma forma de abstração dentro de uma hierarquia de herança, as interfaces oferecem outra maneira de definir contratos em C#. Conforme observa Liberty (2005), uma interface é essencialmente um contrato que garante como uma classe ou uma estrutura irá se comportar.

Em C#, uma interface é um tipo completamente abstrato, declarado com a palavra-chave **interface**, que contém definições de métodos, propriedades, eventos ou indexadores, mas sem qualquer implementação concreta. Ao dizer que uma classe implementa uma interface, estamos afirmando que essa classe concorda em fornecer implementações para todos os membros definidos na interface,

ou seja, em cumprir fielmente o contrato especificado por ela. Por exemplo, poderíamos definir uma interface **IForma** para estipular que qualquer forma deve saber calcular área, semelhantemente ao caso anterior:

```
public interface IForma
{
    double CalcularArea();
}
```

A interface **IForma** declara apenas a assinatura do método **CalcularArea**, sem implementação. Uma classe que implemente **IForma** deverá fornecer o corpo de **CalcularArea**. Se declararmos:

```
public class Triangulo : IForma
{
    public double Base { get; set; }
    public double Altura { get; set; }

    public Triangulo(double b, double h) {
        Base = b;
        Altura = h;
    }

    public double CalcularArea() {
        return (Base * Altura) / 2.0;
    }
}
```

Agora **Triangulo** é uma classe concreta que implementa **IForma**. O compilador força a existência do método **CalcularArea()** com exatamente a assinatura definida na interface, caso contrário a classe não compila (em C#, se faltar implementar algum membro da interface, a classe permanece incompleta e será tratada como **abstract** pelo compilador). Uma vez que **Triangulo** implementa **IForma**, podemos utilizá-lo por meio do tipo da interface:

```
IForma fig = new Triangulo(3, 4);
Console.WriteLine( fig.CalcularArea() ); // Saída: 6
```

Aqui, a variável **fig** é do tipo da interface **IForma**, mas guarda um objeto concreto **Triangulo**. Novamente vemos o princípio do polimorfismo em ação: não importa qual a classe concreta, enquanto ela cumprir o contrato **IForma**, podemos atribuí-la a uma referência de interface e invocar **CalcularArea()** de forma genérica.

Outras classes completamente distintas – por exemplo, poderíamos ter uma classe **Circulo** implementando **IForma** – também podem ser tratadas da mesma maneira. As interfaces permitem essa flexibilidade porque, diferentemente da herança de classes, uma classe pode implementar múltiplas interfaces. Assim, uma mesma classe pode aderir a diversos contratos. Por exemplo, poderíamos ter uma interface **IObjetoDesenhavel** separada, e uma classe **Circulo** poderia implementar tanto **IForma** quanto **IObjetoDesenhavel** se fizesse sentido, combinando contratos de área e de desenho gráfico.

É importante entender as diferenças e os papéis complementares de classes abstratas e interfaces no design de software. Ambas estabelecem contratos, mas de maneiras diferentes.

Classes abstratas fazem parte de uma hierarquia de herança, normalmente representam uma generalização de um grupo de classes relacionadas. Elas podem fornecer implementação comum (código reutilizável) e também definir membros abstratos a serem implementados nas derivadas. Entretanto, cada classe concreta só pode herdar de uma classe base (limitação da herança simples em C#). Isso significa que a escolha de usar uma classe abstrata força um relacionamento hierárquico, por exemplo, **Circulo** é uma **Forma**, **ContaPoupanca** é um tipo de **ContaBancaria** abstrata etc.

Já as interfaces não implicam relação hierárquica é-um, mas sim definem capacidades que podem ser agregadas a qualquer classe, independentemente de onde ela esteja na hierarquia. Para fim de exemplificação, podemos ter uma interface **IImprimivel** que define um contrato para objetos que possam ser impressos em papel. Uma classe **Relatorio** que já herda de uma classe base **Documento** (hipotética) pode implementar **IImprimivel** para sinalizar que sabe se imprimir; ao mesmo tempo, uma classe completamente diferente, digamos **Fotografia**, que herda de **Imagem**, também poderia implementar **IImprimivel**. Não existe um parentesco entre **Relatorio** e **Fotografia**, elas compartilham apenas a capacidade comum definida pela interface. Assim, as interfaces permitem formas de polimorfismo horizontal, no qual classes distintas compartilham comportamentos através de contratos comuns, sem estarem na mesma cadeia de herança.

Até recentemente, havia diferenças claras em termos de implementação: interfaces em C# não podiam fornecer nenhum código, pois todos os métodos de uma interface eram implicitamente abstratos (sem corpo) e públicos. As classes que implementassem a interface tinham de fornecer todo o código. Classes abstratas, por sua vez, podiam ter métodos com implementação concreta, permitindo fatorar código comum.

Contudo, versões mais novas do C# introduziram alguns recursos adicionais às interfaces. A partir do C# 8, por exemplo, interfaces podem conter implementações padrão em métodos (usando a palavra-chave `default` em métodos na interface), assim como também podem declarar membros estáticos, e até membros abstratos estáticos (como em C# 11) para certos cenários avançados.

Esses recursos tornam as interfaces um pouco mais semelhantes a classes abstratas em termos de poder ter alguma implementação. Ainda assim, existem limitações, como no caso de interfaces que não podem ter campos de instância para armazenar estado, apenas constantes ou, mais recentemente, propriedades estáticas `readonly`. Dessa forma, classes abstratas continuam sendo a ferramenta adequada quando se deseja compartilhar estado ou código interno entre classes relacionadas, enquanto interfaces são mais indicadas para definir apenas contratos que qualquer classe pode adotar.

A ocultação de detalhes e a abstração andam de mãos dadas: ao programar com classes abstratas e interfaces, o desenvolvedor se concentra nas operações disponibilizadas (o contrato) e não nas minúcias de como cada classe as realiza. Essa abordagem promove um acoplamento fraco entre os componentes do sistema, de forma que os módulos comunicam-se através de interfaces bem definidas, podendo mudar internamente sem impactar uns aos outros.

A utilização de classes e membros abstratos permite criar **modelos genéricos de comportamento**, deferindo a implementação específica para pontos apropriados da hierarquia de herança.

Já as interfaces, vistas como **contratos puros**, permitem estabelecer conjuntos de funcionalidades que várias classes (mesmo de linhagens diferentes) se comprometem a fornecer.

Em conjunto, esses mecanismos elevam o nível de abstração do código e implementam o princípio do programar voltado à interface, não à implementação. Assim, sistemas bem projetados expõem apenas interfaces necessárias e escondem detalhes internos, facilitando a manutenção e a evolução do software sem quebrar contratos externos.

O resultado são programas mais seguros, robustos e adaptáveis, nos quais cada classe desempenha um papel claro e cumpre os contratos esperados, e onde mudanças nas entranhas de uma classe não repercutem indevidamente em outras partes do sistema.

3.2 Interfaces, implementação explícita, substituíbilidade (LSP) com exemplos

Uma interface define um contrato que classes ou structs devem satisfazer por meio da implementação de determinados membros (métodos, propriedades, indexadores ou eventos).

Diferentemente de classes abstratas, interfaces não possuem estado e tradicionalmente não contêm implementação, elas especificam apenas o que deve ser feito, mas não como. Uma classe que assina esse contrato (diz-se que implementa a interface) compromete-se a fornecer código para todos os membros definidos pela interface.

Por exemplo, imagine uma interface `IFormaGeometrica` com um método `CalcularArea()`. Qualquer classe que implemente `IFormaGeometrica` deverá fornecer sua própria versão de `CalcularArea()`. Isso permite que instâncias de classes distintas sejam tratadas de maneira uniforme: um algoritmo que opera sobre `IFormaGeometrica` pode receber tanto um objeto `Circulo` quanto um `Retangulo`, desde que ambos implementem a interface, garantindo, assim, a **substituíbilidade polimórfica** esperada em orientação a objetos.

Em suma, a interface define um conjunto de funcionalidades esperadas e as classes concretas cumprem esse contrato, promovendo um **baixo acoplamento** e um design mais flexível do software. Vale mencionar que, a partir do C# 8.0, interfaces passaram a suportar implementações padrão de métodos, o que permite evoluir uma interface sem quebrar código existente; entretanto, essa funcionalidade não altera o papel fundamental da interface como um contrato que as classes devem honrar.

A seguir, apresentamos uma interface simples e duas classes que a implementam. A interface `IFormaGeometrica` define um contrato para formas geométricas que podem ter sua área calculada. As classes `Retangulo` e `Circulo` implementam essa interface, cada uma fornecendo a lógica específica para calcular sua área.

```
interface IFormaGeometrica {
    double CalcularArea();
}

class Retangulo : IFormaGeometrica {
    public double Largura { get; set; }
    public double Altura { get; set; }

    public double CalcularArea() {
        return Largura * Altura;
    }
}

class Circulo : IFormaGeometrica {
    public double Raio { get; set; }

    public double CalcularArea() {
        return Math.PI * Raio * Raio;
    }
}
```

Nesse código, **Retangulo** e **Circulo** oferecem implementações distintas do método **CalcularArea()**. Agora, graças ao polimorfismo proporcionado pelas interfaces, podemos escrever um método que calcula a área total de diversas formas geométricas sem nos preocuparmos com as classes concretas de cada uma, como vemos a seguir:

```
double AreaTotal(IFormaGeometrica[] formas) {
    double soma = 0;
    foreach (IFormaGeometrica forma in formas) {
        soma += forma.CalcularArea();
    }
    return soma;
}

// Uso do método:
var formas = new IFormaGeometrica[] {
    new Retangulo { Largura = 5, Altura = 4 }, // área = 20
    new Circulo { Raio = 3 }                  // área ≈ 28.27
};
Console.WriteLine(AreaTotal(formas)); // Saída esperada ≈ 48.27
```

No exemplo, o array **formas** armazena instâncias de **Retangulo** e **Circulo** através do tipo da interface **IFormaGeometrica**. O método **AreaTotal** itera por cada elemento e invoca **CalcularArea()** sem precisar saber se a forma é um retângulo ou um círculo; isso demonstra a substituíbilidade polimórfica em ação. Assim, qualquer objeto que implemente **IFormaGeometrica** pode ser usado no lugar de outro, respeitando-se o contrato da interface. Esse é exatamente o comportamento descrito pelo **princípio da substituição de Liskov** (LSP), abordado mais adiante.

Em C#, o modo padrão de uma classe implementar os membros de uma interface é incluí-los em sua definição com modificador de acesso público (**implementação implícita**). Entretanto, a linguagem oferece a possibilidade de implementação explícita de interface, em que a classe fornece a implementação de um membro de interface sem expô-lo em seu contrato público convencional. Em

uma implementação explícita, o nome do membro da interface é qualificado com o nome da interface. Dessa forma, o membro só pode ser acessado quando a instância do objeto é referenciada através do tipo da interface, e não através do tipo concreto da classe.

As implementações explícitas são úteis em alguns cenários específicos. Um caso comum é quando uma classe implementa múltiplas interfaces que possuem métodos homônimos ou semanticamente distintos. Considere o exemplo a seguir, no qual uma mesma classe implementa duas interfaces diferentes com membros de mesmo nome:

```
interface IGreetingsEn {
    string Saudar();
}
interface IGreetingsFr {
    string Saudar();
}

class CumprimentoMulti : IGreetingsEn, IGreetingsFr {
    // Implementação implícita para inglês:
    public string Saudar() {
        return "Hello"; // Saudação em inglês por padrão
    }
    // Implementação explícita para francês:
    string IGreetingsFr.Saudar() {
        return "Bonjour"; // Saudação em francês somente via interface
    }
}
IGreetingsFr
}
```

No exemplo, `CumprimentoMulti` fornece duas versões do método `Saudar()`: uma pública (implícita) retornando **“Hello”** e outra explícita (associada à interface francesa) retornando **“Bonjour”**. O efeito é o seguinte:

```
var obj = new CumprimentoMulti();
Console.WriteLine(obj.Saudar()); // Saída: Hello
Console.WriteLine(((IGreetingsEn)obj).Saudar()); // Saída: Hello (IGreetingsEn
usa implícita)
Console.WriteLine(((IGreetingsFr)obj).Saudar()); // Saída: Bonjour
(IGreetingsFr usa explícita)
```

Quando chamamos `obj.Saudar()` diretamente, estamos utilizando a versão pública do método (implementação em inglês). Já ao converter o objeto para `IGreetingsFr` e invocar `Saudar()`, executa-se a implementação explícita definida para o francês. Note que, sem o cast para `IGreetingsFr`, não é possível chamar a versão francesa, pois ela não faz parte da interface pública normal do objeto `CumprimentoMulti`.

Em tempo de compilação, `obj.Saudar()` se vincula à implementação pública (inglês), e apenas referenciando o objeto pelo tipo `IGreetingsFr` acessamos a outra versão. Esse comportamento ilustra a essência da implementação explícita: segregar diferentes comportamentos para diferentes interfaces que uma classe implementa, evitando conflitos de nomes e controlando a visibilidade de cada membro.

O LSP é o L do acrônimo SOLID, um conjunto clássico de princípios de design de software. Formulado originalmente por Barbara Liskov em 1988, o LSP propõe que, se S é um subtipo de T, então os objetos do tipo T em um programa podem ser substituídos por objetos do tipo S sem que seja necessário alterar o comportamento correto desse programa.

Em termos mais simples, qualquer instância de uma classe derivada deve poder ser usada no lugar de uma instância da classe base (ou interface), e o programa deve continuar funcionando adequadamente, preservando as expectativas de comportamento.

Esse princípio está intimamente ligado à ideia de contrato: a classe derivada (ou classe que implementa uma interface) deve honrar todas as garantias e expectativas estabelecidas pela classe base (ou interface), não introduzindo restrições adicionais nem comportamentos inesperados do ponto de vista do código cliente.

Para entender o LSP, considere um exemplo clássico de violação desse princípio: o problema do **Quadrado** e do **Retângulo**. Suponha que temos uma classe base **Retângulo** com propriedades de largura e altura, e métodos para ajustar essas dimensões. Agora, imagine uma classe **Quadrado** derivada de **Retângulo**.

À primeira vista, parece razoável, afinal, matematicamente, todo quadrado é um retângulo com lados de mesmo comprimento. Entretanto, ao modelar isso em código, surgem problemas sutis, como vemos na implementação hipotética a seguir:

```
class Retangulo {
    public virtual void DefinirLargura(double larg) { Largura = larg; }
    public virtual void DefinirAltura(double alt) { Altura = alt; }
    public double Largura { get; protected set; }
    public double Altura { get; protected set; }
    public virtual double Area => Largura * Altura;
}

class Quadrado : Retangulo {
    public override void DefinirLargura(double larg) {
        base.DefinirLargura(larg);
        base.DefinirAltura(larg);
    }
    public override void DefinirAltura(double alt) {
        base.DefinirAltura(alt);
        base.DefinirLargura(alt);
    }
}
```

Nesse código, **Quadrado** sobrescreve os métodos de definição de largura e altura de modo que ambos os lados permaneçam iguais (isto é, caso se defina a largura de um quadrado para 5, a altura também se torna 5, e vice-versa). Essa implementação parece fazer sentido para manter a consistência geométrica do quadrado, mas vejamos do ponto de vista de quem usa essas classes:

```
void AumentarRetanguloEmDobro(Retangulo r) {
    r.DefinirLargura(r.Largura * 2);
    r.DefinirAltura(r.Altura * 2);
    // Esperamos que a área agora seja quatro vezes maior:
    Console.WriteLine("Área quadruplicada: " + r.Area);
}

// Cenário de uso:
Retangulo meuRet = new Retangulo();
meuRet.DefinirLargura(5);
meuRet.DefinirAltura(10);
// Área inicial = 50
AumentarRetanguloEmDobro(meuRet);
// Área esperada = 200 (50 * 4) -> correto para Retangulo

Retangulo meuQuad = new Quadrado();
meuQuad.DefinirLargura(5);
meuQuad.DefinirAltura(10);
// Aqui, ao definir altura para 10, a largura também se torna 10 (Quadrado impõe
// lados iguais)
// Área inicial de meuQuad = 100 (10*10)
AumentarRetanguloEmDobro(meuQuad);
// O método AumentarRetanguloEmDobro presume área quadruplicada,
// mas para Quadrado a área resultante será 400 (pois os lados dobraram de 10
// para 20, área 20*20)
```

No exemplo, **AumentarRetanguloEmDobro** foi escrito pensando em um retângulo genérico: dobrar largura e altura deveria multiplicar a área por quatro. De fato, com um **Retangulo** comum de 5 x 10 (área 50), o procedimento resulta em um retângulo de 10 x 20, com área 200, quadruplicando a área conforme esperado.

Entretanto, ao passar um **Quadrado** (derivado de **Retangulo**) para essa mesma função, o comportamento diverge: após definir largura = 5 e altura = 10, o objeto **Quadrado** acaba com largura = 10 e altura = 10 (porque, ao definir a altura, a implementação de **Quadrado** também ajusta a largura). Ou seja, antes mesmo de entrar no método, o retângulo (na verdade, um quadrado) já não tem as dimensões esperadas de 5 x 10, mas sim 10 x 10, com área 100. Quando o método tenta dobrar cada dimensão, o quadrado passa a 20 x 20, com área 400 – oito vezes maior que a área inicial pretendida de 50, em vez de quatro vezes.

O código cliente **AumentarRetanguloEmDobro** fica quebrado para o subtipo **Quadrado**, violando claramente o princípio da substituíbilidade (LSP). Em outras palavras, **Quadrado** é ainda um **Retangulo** em termos de herança de tipos, mas não se comporta como um retângulo genérico do ponto de vista de quem usa, uma vez que não permite tratar largura e altura de forma independente. Esse exemplo ilustra que nem toda relação é-um na modelagem conceitual se traduz em uma herança que respeita o LSP.

A solução para esse dilema seria repensar a hierarquia de classes. Poderíamos, por exemplo, não fazer **Quadrado** herdar de **Retangulo**, mantendo as classes separadas ou usando composição.

O importante é garantir que métodos que esperam um **Retângulo** possam receber um **Quadrado** sem surpresas, ou seja, respeitar o LSP. De modo geral, para aderir ao LSP, tipos derivados ou implementações de interfaces devem preservar as expectativas comportamentais do tipo base. Isso inclui manter válidas todas as afirmações lógicas (invariantes) do tipo base e não fortalecer restrições.

Por exemplo, se uma interface **IRepositorio** define um método `Remover(item)` que, em sua documentação, promete remover um item se este existir, então uma classe que implemente essa interface não deveria simplesmente lançar uma exceção não suportada ao tentar remover, pois isso quebraria a expectativa do usuário da interface.

Da mesma forma, se uma classe derivada sobrescreve um método, ela não deve exigir condições mais estritas para operá-lo do que a classe base exigia (pré-condições) nem prover resultados menos corretos do que a base prometia (pós-condições).

Em suma, os consumidores de um tipo não devem precisar se preocupar se o objeto concreto fornecido é da classe base ou de uma classe derivada – o comportamento observado deve ser consistente com o contrato esperado.

Quando combinamos o conceito de interfaces com o LSP, percebemos que uma interface bem projetada ajuda naturalmente a cumprir a substituíbilidade. Afinal, interfaces explicitam um conjunto de operações esperadas, e qualquer classe que as implemente deve fazê-lo de maneira coerente. Se uma classe implementa uma interface, mas quebra alguma expectativa básica de seus métodos, ela estará violando o LSP.

Considere, por exemplo, uma interface simples **IContaBancaria** com um método `Sacar(decimal valor)` que deve debitar uma quantia do saldo. Se uma implementação concreta de **IContaBancaria** permitir `Sacar` apenas valores até certo limite sem lançar exceção, isso deve estar documentado e fazer parte das expectativas do contrato, caso contrário, o código que use polimorficamente **IContaBancaria** poderia falhar inesperadamente com certas implementações.

Assim, ao projetar interfaces, é importante definir claramente (em documentação ou contratos) quais comportamentos são garantidos, para que as classes derivadas não introduzam surpresas. Na prática, o LSP serve como um guarda-chuva conceitual que engloba vários outros princípios: respeitá-lo significa que você está também aplicando corretamente os princípios de abstração e de design por contrato.

Um ponto interessante é que a própria técnica de implementação explícita de interfaces deve ser usada com critério para não ferir a substituíbilidade. Implementar um membro de forma explícita, tornando-o inacessível via referência de classe concreta, não deve significar omitir ou invalidar esse comportamento quando o objeto é tratado polimorficamente.

Por exemplo, se uma classe implementa uma interface somente para cumprir um contrato em cenários específicos, mas tenta esconder completamente essa funcionalidade do uso geral, devemos nos questionar se essa classe deveria realmente ser do tipo da interface.

Em alguns casos, esconder um membro da interface com implementação explícita pode indicar que a classe não pretende oferecer aquela capacidade de forma plena, o que sugere um possível design inadequado em face do LSP.

Interfaces, implementação explícita e substituíbilidade estão inter-relacionadas na escrita de código orientado a objetos robusto e flexível. As interfaces fornecem uma maneira de definir contratos formais, permitindo que diferentes classes sejam intercambiáveis a partir de um tipo comum: a base do polimorfismo em C#.

A implementação explícita de interfaces é uma funcionalidade da linguagem que oferece controle adicional de encapsulamento, possibilitando resolver conflitos de nomenclatura e organizar melhor as exposições de métodos de uma classe quando múltiplos contratos estão em jogo.

Já o LSP lembra que não basta uma classe assinar a interface ou herdar de uma classe base: ela deve realmente se comportar como o tipo que promete ser, de forma coerente e sem surpreender os clientes. Seguir o LSP leva a designs mais confiáveis, pois quando qualquer instância de uma implementação pode substituir outra, podemos estender sistemas com novas classes sem medo de quebrar código existente, aproveitando o máximo benefício da abstração.

Por fim, ao definir interfaces claras e aderir estritamente aos contratos (respeitando invariantes, pré e pós-condições), garantimos substituíbilidade. Além disso, ao usar cuidadosamente ferramentas como a implementação explícita, podemos aumentar a elegância do design sem sacrificar a corretude polimórfica. Esses princípios, aplicados em conjunto, resultam em um código C# mais manutenível, extensível e alinhado aos fundamentos da orientação a objetos, metas centrais do desenvolvimento de software de qualidade.

4 HERANÇA E POLIMORFISMO

Em desenvolvimento de software, a POO costuma ser apresentada como uma solução para organizar sistemas complexos em termos de tipos, responsabilidades e colaborações. Porém, quando o programa cresce, começam a aparecer sintomas incômodos: mudanças simples exigem alterações em muitos arquivos, correções de defeitos introduzem novos erros em pontos inesperados e a equipe passa a sentir receio de mexer em certas partes do código.

Esses problemas não surgem apenas por falta de boa vontade dos programadores, mas estão ligados a decisões estruturais sobre como as classes se relacionam, quais extensões foram previstas desde o início e até que ponto o código aceita novas funcionalidades sem se tornar frágil.

Nesse cenário, a herança, as classes abstratas, as classes seladas, a composição e o polimorfismo não são apenas recursos de sintaxe de linguagens como C#, mas instrumentos de projeto que afetam diretamente a qualidade do software.

O uso descuidado de herança tende a produzir hierarquias rígidas, difíceis de alterar, nas quais mudanças em uma superclasse podem causar efeitos colaterais em série nas subclasses. Ao mesmo

tempo, insistir em estruturas puramente planas, sem generalização alguma, leva à duplicação de código e à perda de expressividade. A questão central deixa de ser usar ou não herança, passando a ser em que situações a herança é adequada, quando ela começa a gerar dependências perigosas e que alternativas estão disponíveis dentro da própria linguagem.

Como exemplo didático, o desenvolvimento de jogos é um terreno fértil para observar essas tensões. Personagens, armas, efeitos, inimigos e objetos de cenário compartilham comportamentos e propriedades, mas também variam de maneiras sutis, que mudam ao longo do tempo de produção do jogo. Uma escolha apressada de hierarquia pode dificultar a introdução de novos tipos de personagens ou a combinação de habilidades sem refatorações pesadas, o que atrasa o fluxo de trabalho e aumenta a chance de regressões.

Nesse contexto, a composição por meio de componentes, o uso criterioso de classes abstratas e seladas e o emprego de interfaces para expressar capacidades comuns surgem como resposta às necessidades práticas, como manter o código extensível, reduzir o acoplamento entre partes do sistema e controlar o impacto de mudanças em mecanismos de jogo já implementados.

A seguir, você será inserido justamente nesse conjunto de preocupações, examinando as ferramentas oferecidas por C# para construir modelos de domínio mais flexíveis e menos suscetíveis a efeitos colaterais inesperados. A discussão não interessa apenas a quem está aprendendo a sintaxe da linguagem, mas a quem precisa tomar decisões de projeto em sistemas que continuarão evoluindo, seja em jogos ou em outras aplicações interativas.

4.1 Hierarquias, **abstract/sealed**, composição como alternativa

Em POO, é comum organizar as classes em hierarquias de herança para modelar relações do tipo é-um entre entidades. Por exemplo, em um jogo poderíamos ter uma classe base genérica **Personagem** da qual derivam classes específicas como **Guerreiro** ou **Arqueiro**.

Herdar de uma classe base permite que as subclasses reutilizem atributos e métodos comuns, enquanto especializam comportamentos específicos. Essa reutilização por meio de herança é chamada de reuso white-box (caixa-branca), pois a subclasse tem visibilidade sobre a implementação interna da superclasse.

Em outras palavras, a subclasse abre a caixa da classe base e aproveita seu código, podendo alterar comportamentos por meio de métodos sobrescritos (**override**). A herança facilita o polimorfismo: um método que recebe um **Personagem** pode aceitar tanto um **Guerreiro** quanto um **Arqueiro**, por exemplo, tratando-os de forma genérica. Assim, herança e hierarquias proporcionam uma forma natural de expressar generalizações e especializações (relações é-um) no projeto de software.

No entanto, a herança deve ser utilizada com critério. Uma hierarquia muito profunda ou mal planejada pode tornar o código rígido e frágil diante de mudanças (o chamado *fragile base class problem*). Alterações em classes base podem afetar todas as subclasses, e nem sempre é fácil estender funcionalidade sem modificar código existente.

Além disso, o mecanismo de herança simples do C#, como em Java, não permite herança múltipla de classes. Assim, cada classe pode ter apenas uma superclasse direta. Isso significa que, quando se tenta modelar um conceito que naturalmente teria múltiplos pais, é preciso recorrer a outras técnicas, como interfaces ou composição.

Uma classe abstrata é uma classe pensada para ocupar o topo ou o meio de uma hierarquia, definindo um conceito genérico que não pode ser instanciado diretamente. Como já vimos, tais classes frequentemente contêm métodos abstratos, que são declarações de métodos sem implementação, obrigando as subclasses concretas a fornecer uma implementação.

Por exemplo, poderíamos declarar `public abstract class Personagem` com um método abstrato `public abstract void Atacar()`; . As subclasses `Guerreiro` e `Arqueiro` seriam então obrigadas a implementar esse método `Atacar` de formas específicas (por exemplo, o guerreiro desferindo um golpe de espada, o arqueiro atirando flechas). Classes abstratas permitem fatorar comportamento comum e definir um contrato para as subclasses, garantindo que certas operações existam em todas as especializações.

Vale notar que uma classe abstrata ainda pode conter implementação concreta de alguns métodos e propriedades, compartilhando código entre as derivadas. Ela serve, portanto, como um modelo parcial com lacunas a serem preenchidas pelas subclasses. A tentativa de instanciar uma classe abstrata (por exemplo, fazer `new Personagem()`) resulta em erro de compilação, sendo necessário instanciar uma das subclasses concretas.

A seguir, apresenta-se um exemplo simplificado ilustrando uma hierarquia de herança no domínio de um jogo. Definimos uma classe abstrata `Personagem` com um método abstrato `Atacar()`, e duas subclasses concretas `Guerreiro` e `Arqueiro` que implementam o ataque de formas distintas:

```
using System;

abstract class Personagem {
    public abstract void Atacar();
}

class Guerreiro : Personagem {
    public override void Atacar() {
        Console.WriteLine("O guerreiro ataca com sua espada!");
    }
}

class Arqueiro : Personagem {
    public override void Atacar() {
        Console.WriteLine("O arqueiro dispara uma flecha!");
    }
}

class Program {
    static void Main() {
        Personagem jogador1 = new Guerreiro();
    }
}
```

```
Personagem jogador2 = new Arqueiro();
jogador1.Atacar(); // Saída: O guerreiro ataca com sua espada!
jogador2.Atacar(); // Saída: O arqueiro dispara uma flecha!
}
```

Nesse código, **Guerreiro** e **Arqueiro** herdam de **Personagem** e cada um define sua própria lógica para **Atacar()**. O polimorfismo permite invocar **Atacar()** genericamente em um **Personagem**, dessa forma, em tempo de execução, o método apropriado à subclasse é chamado.

Em contraste com as classes abstratas (que servem apenas como base), a linguagem C# também permite marcar classes de forma a impedir que outras herdem delas. Usa-se a palavra-chave **sealed** para indicar que uma classe não pode ter subclasses. É equivalente à classe **final** da linguagem Java.

Por exemplo, se declararmos **sealed class Arqueiro : Personagem { ... }**, o compilador não permitirá derivar nenhuma nova classe de **Arqueiro**. O objetivo de uma classe selada é sinalizar que sua hierarquia termina ali, seja por motivos de design (a classe representa uma entidade específica que não deve ser estendida inadvertidamente) ou por motivos de segurança e desempenho.



Observação

Muitas classes do .NET Framework são seladas, como **System.String**, que é selada para garantir sua semântica de imutabilidade e porque não faria sentido haver subclasses de **string**.

Marcar uma classe como selada pode prevenir herança acidental ou indevida. Se um desenvolvedor tentar herdar de uma classe selada, obterá um erro em tempo de compilação. Em cenários de projeto, uma classe pode ser selada quando o designer do código quer evitar que ela seja estendida, seja para manter controle sobre seu comportamento, seja porque foi concebida para não ser alterada via herança.

Como benefício adicional, métodos virtuais de uma classe selada podem ser internamente otimizados pelo runtime, já que não há a necessidade de considerar futuras substituições (**override**) em subclasses inexistentes.

Além de classes inteiras, C# permite selar métodos virtuais individuais. Uma subclasse pode sobrescrever um método virtual da base e marcá-lo como **sealed override**, indicando que mais nenhuma classe derivada poderá sobrescrevê-lo novamente. Essa técnica é menos comum, mas útil para congelar personalizações em um determinado nível da hierarquia.

A **composição de objetos** é outra forma fundamental de reutilização e organização de código. Em vez de estabelecer uma relação hierárquica é-um, a composição estabelece uma relação **tem-um** (has-a). Isso significa que um objeto é construído a partir de outros objetos internos, delegando a eles parte de sua funcionalidade.

Diferentemente da herança, na composição não há relação de especialização, nem compartilhamento direto de implementação, já que cada classe composta mantém sua individualidade e expõe apenas o que for necessário via sua interface pública. Por essa razão, costuma-se chamar a composição de reuso black-box (caixa-preta), pois o objeto complexo trata seus componentes como caixas-pretas, utilizando-os através de suas interfaces, sem se acoplar aos detalhes internos deles.

Em resumo, objetos complexos podem ser construídos agregando objetos menores; essa agregação modela características do mundo real (por exemplo, um carro tem-um motor, tem-um chassi, tem-um sistema elétrico etc.), promovendo um design modular.

Uma diretriz clássica de design orientado a objetos, apresentada por Gamma *et al.* (2005), é preferir composição em vez de herança sempre que possível. Essa recomendação reconhece que a herança, embora poderosa, pode levar a sistemas rígidos e fortemente acoplados, enquanto a composição favorece maior flexibilidade e menor acoplamento.

A composição torna mais fácil modificar comportamentos em tempo de execução, trocando os objetos componentes, em comparação com a herança, que fixa o comportamento no momento da definição da subclasse. Além disso, a composição evita alguns problemas da herança: não sofre das limitações de herança simples (um objeto pode ter múltiplos componentes de tipos diferentes, contornando a falta de herança múltipla) e reduz o risco de efeitos colaterais de mudanças em superclasses.

No contexto de desenvolvimento de jogos, por exemplo, a composição é amplamente utilizada através do padrão de entidade-componente. Em vez de modelar uma infinidade de tipos de personagens e objetos do jogo em uma enorme árvore de herança, muitas engines adotam o princípio de compor entidades a partir de componentes modulares.

A título ilustrativo, uma entidade genérica de jogo pode ter componentes separados para comportar física, renderização, comportamento de IA, habilidades etc. Assim, em vez de criar subclasses para cada combinação possível (por exemplo, `InimigoVoadorAtirador` herda de `InimigoVoador`, que herda de `Inimigo` etc.), monta-se um objeto `Inimigo` adicionando-lhe um componente `Voo` e um componente `AtaqueDistancia`. Essa abordagem composicional tende a produzir sistemas mais flexíveis e fáceis de estender, como novas habilidades que podem ser adicionadas criando novos componentes e anexando-os a entidades, sem a necessidade de alterar hierarquias existentes.

No exemplo anterior, utilizamos herança para diferenciar o ataque de um guerreiro e de um arqueiro. Agora, vamos supor que queiramos permitir que um personagem mude de arma durante o jogo, ou seja, que determinado guerreiro do jogo pode largar sua espada e pegar um arco.

Com a solução baseada somente em herança, não é trivial fazer isso: teríamos que instanciar um novo objeto **Arqueiro** a partir do antigo **Guerreiro** (ou talvez ter atributos opcionais), o que quebra a naturalidade do modelo orientado a objetos, pois um guerreiro deixaria de ser **Guerreiro** para virar **Arqueiro** dinamicamente, algo não representado pela hierarquia original.

A composição oferece uma alternativa elegante: em vez de subclasses para cada tipo de ataque, usamos objetos "arma" que podem ser acoplados ou desacoplados de um personagem em tempo de execução. O personagem continua sendo um objeto de uma única classe, mas seu comportamento de ataque varia conforme o objeto arma que ele possui no momento.

A seguir, reimplementamos o cenário usando composição. Criamos uma classe **Arma** (abstrata), da qual derivam **Espada** e **Arco**, representando diferentes tipos de arma com suas lógicas de ataque. A classe **Personagem** agora não precisa ser abstrata nem ter subclasses para cada tipo, ela tem um campo do tipo **Arma** e delega a ela a ação de atacar. Assim, trocar a arma do personagem muda seu ataque sem ser preciso trocar o objeto **Personagem** em si.

```
using System;

abstract class Arma {
    public abstract void Atacar();
}

class Espada : Arma {
    public override void Atacar() {
        Console.WriteLine("Golpe de espada!");
    }
}

class Arco : Arma {
    public override void Atacar() {
        Console.WriteLine("Disparo de flecha!");
    }
}

class Personagem {
    public Arma ArmaAtual { get; set; }
    public Personagem(Arma armaInicial) {
        this.ArmaAtual = armaInicial;
    }
    public void Atacar() {
        if (ArmaAtual != null)
            ArmaAtual.Atacar();
        else
            Console.WriteLine("Personagem desarmado não pode atacar.");
    }
}

class Program {
    static void Main() {
        Personagem heroi = new Personagem(new Espada());
        heroi.Atacar();           // Saída: Golpe de espada!
        heroi.ArmaAtual = new Arco();
        heroi.Atacar();           // Saída: Disparo de flecha!
    }
}
```

Nesse código, a classe **Personagem** compõe um objeto do tipo **Arma**. Inicialmente o herói é criado com uma **Espada**. Ao chamar `herói.Atacar()`, a chamada é delegada para `Espada.Atacar()`, produzindo a mensagem de ataque com espada. Em seguida, mudamos a composição do objeto em tempo de execução: atribuímos uma instância de **Arco** à propriedade `ArmaAtual` do herói. Agora, `herói.Atacar()` resultará na ação definida em `Arco.Atacar()`, ou seja, disparar uma flecha.

Note que todo esse comportamento foi obtido sem criar subclasses de **Personagem**, já que a classe do herói permanece a mesma e apenas a arma (um componente) mudou. Isso ilustra o poder da composição: flexibilizar a variação de comportamentos e atributos de um objeto sem explodir a hierarquia de classes.

O exemplo também mostra que a composição pode trabalhar em conjunto com outros conceitos, como interfaces ou classes abstratas para os componentes, onde, no caso, **Arma** é abstrata e **Espada/Arco** são implementações concretas.

A composição e a herança não se excluem mutuamente, de fato, podem complementar uma à outra. Por exemplo, poderíamos ter uma hierarquia de armas (arma corpo a corpo, arma de longa distância etc.) combinada com diferentes personagens, ou personagens organizados em hierarquia usando componentes intercambiáveis.

Outrossim, a herança é indicada quando há uma clara relação conceitual de especialização e quando se deseja aproveitar ou estender a implementação de uma classe existente. Ela provê uma estrutura rígida, porém às vezes conveniente, especialmente quando combinada com polimorfismo para tratar coleções de objetos relacionados de forma uniforme. Entretanto, deve-se evitar hierarquias muito profundas ou complexas demais. Problemas clássicos da orientação a objetos, como a explosão combinatória de subclasses (quando precisamos criar subclasses para cada combinação de aspectos variantes), são um sinal de alerta de que talvez a composição fosse uma escolha melhor.

Já a composição brilha em cenários de composição dinâmica de comportamentos e alta flexibilidade. Novos requisitos podem ser atendidos adicionando ou substituindo componentes, em vez de alterando árvores de herança. O acoplamento tende a ser menor, de forma que cada classe se concentra em uma responsabilidade específica e é usada por outras por meio de interfaces bem-definidas (princípios da responsabilidade única e da inversão de dependência, alinhados com o design SOLID).

Como ponto de atenção, a composição às vezes exige escrever um pouco mais de código "encaminhador" (por exemplo, métodos que simplesmente chamam internamente métodos do componente), e entender a interação de múltiplos objetos pode ser mais complexo do que seguir uma hierarquia linear. Além disso, nem todos os relacionamentos conceituais ficam naturais com composição: se a relação é verdadeiramente de especialização, usar herança pode tornar o código mais claro.

Em projetos reais, é comum usar ambas as abordagens. A literatura sugere privilegiar a composição quando há dúvida, exatamente por ela oferecer caminhos de evolução mais flexíveis. Porém, a herança continua sendo uma ferramenta valiosa, principalmente para modelar abstrações e permitir polimorfismo

básico de forma simples. Por esse motivo, o importante é o desenvolvedor compreender as forças e as fraquezas de cada técnica.

Conforme observa Weisfeld (2013), tanto a herança quanto a composição são técnicas essenciais no desenvolvimento OO, e o uso adequado de cada uma, no contexto correto, é o que leva a um projeto mais robusto e reutilizável.

As hierarquias de herança fornecem um esqueleto organizacional e contratos abstratos, enquanto a composição oferece substância e flexibilidade para montar sistemas complexos. Assim, encontrar o equilíbrio certo entre ambas é parte da arte do design de software.

4.2 Despacho virtual: **virtual/override**, polimorfismo por interface

Como vimos, o polimorfismo refere-se à capacidade de tratar objetos diferentes de forma uniforme, desde que compartilhem uma característica em comum (por exemplo, um mesmo tipo base ou contrato). Em outras palavras, ele permite que objetos de classes distintas, mas relacionadas, sejam manipulados de maneira genérica, cada um respondendo de forma específica à mesma ação.

Em C#, o polimorfismo se manifesta principalmente de duas formas: através de métodos virtuais (mecanismo de despacho virtual em hierarquias de herança) e através de interfaces (polimorfismo por interface). Ambas as abordagens habilitam o chamado late binding (vinculação tardia), ou seja, a decisão de qual método executar ocorre somente em tempo de execução, dependendo do tipo real do objeto, ainda que o código tenha sido escrito de forma genérica. Dessa maneira, é possível invocar operações em um objeto sem conhecer exatamente sua classe concreta – apenas sabendo que ele cumpre determinada interface polimórfica definida em alguma superclasse ou interface implementada.

Esse recurso aumenta significativamente a flexibilidade e a extensibilidade dos programas orientados a objetos, já que novos tipos podem ser introduzidos no sistema substituindo comportamentos sem a necessidade de modificar o código que invoca esses comportamentos.

A seguir, discutiremos em detalhes as duas formas de polimorfismo em C# mencionadas: via métodos virtuais (com o uso de **virtual/override**) e via interfaces. Vale notar que essas técnicas não são excludentes, elas frequentemente se complementam no design de sistemas em C#, mas serão explicadas separadamente para fins didáticos.

No contexto de herança de classes, a linguagem C# oferece suporte ao polimorfismo através de métodos virtuais e do mecanismo de despacho virtual. Uma classe base pode declarar que determinado método é virtual, indicando que as classes derivadas podem sobrescrevê-lo para alterar seu comportamento. Um método virtual definido na classe base possui, portanto, uma implementação padrão, possivelmente incompleta ou genérica, que poderá ser sobrescrita (**override**) por uma classe derivada.

Em contrapartida, o C# também permite declarar métodos abstratos em classes base. Lembre-se de que um método abstrato é, essencialmente, um contrato que a classe base impõe às subclasses, pois ele

não fornece nenhuma implementação na classe abstrata, apenas a assinatura; consequentemente, toda classe derivada concreta é obrigada a fornecer uma implementação (**override**) para esse método.

Em suma, métodos virtuais e abstratos diferem apenas em possuir ou não uma implementação padrão na classe base, mas ambos fazem parte da chamada interface polimórfica da classe. É através de membros virtuais (eventualmente abstratos) que a classe base define um conjunto de comportamentos que poderão ser variados nas subclasses.

Quando uma classe derivada sobrescreve um membro virtual ou abstrato herdado, ela essencialmente redefine como responder a uma mesma chamada feita em nível de classe base. Assim, objetos de subclasses diferentes podem ser tratados como do tipo da superclasse, executando sua versão particular do método sobrescrito, caracterizando o polimorfismo.



Lembrete

O recurso de métodos virtuais e abstratos do C# alinha-se ao princípio "uma interface, múltiplas implementações", típico da POO.

Por padrão, métodos de instância em classes C# não são polimórficos, isto é, a não ser que sejam declarados como virtuais (ou que estejam implementando um método de interface, como veremos adiante), os métodos de uma classe não podem ser sobrescritos em subclasses.

Lembre-se de que um método não virtual é chamado de método selado (**sealed**) nesse contexto, pois sua implementação fica fechada para modificações polimórficas em herdeiros. Portanto, para permitir que um método seja substituível em subclasses, a classe base deve declará-lo como virtual. Já a classe derivada usa o modificador **override** para indicar que está fornecendo uma nova implementação para um método virtual herdado.

Outra restrição sintática é que métodos virtuais não podem ser declarados como privados nem estáticos, tampouco podem ser marcados como sealed na própria classe base (o selo é o inverso – é aplicado em subclasses para impedir novas sobrescritas).

Além disso, obviamente, um método com **override** não pode alterar a assinatura do método **virtual** correspondente. Essas regras visam manter a consistência do contrato: o método sobrescrito na subclasse deve ter exatamente a mesma assinatura e modificador de acesso do método virtual original (o compilador reforça isso).

Convém notar também que métodos virtuais não podem ser marcados como **abstract** (e vice-versa), pois são conceitos mutuamente exclusivos, já que o método abstrato, por definição, não possui corpo algum. No entanto, um método abstrato implica virtualidade: toda implementação de método abstrato em uma classe derivada é, de fato, uma sobrescrita.

A seguir, apresentamos um exemplo prático para ilustrar o polimorfismo via métodos virtuais e **override**. Suponha um cenário de um jogo no qual existe uma classe base **Character** (Personagem) que define comportamentos genéricos de um personagem do jogo. Essa classe possui um método virtual chamado **Attack()**, que representa o ato de atacar um inimigo. A implementação padrão de **Attack()** na classe base pode ser simples, apenas informando que o personagem ataca de alguma forma genérica.

Em seguida, temos subclasses de **Character** representando tipos específicos de personagens, por exemplo **Warrior** (Guerreiro) e **Mage** (Mago), cada uma sobrescrevendo o método **Attack()** para realizar o ataque de maneira distinta, conforme sua especialização. No código a seguir vemos essa hierarquia de classes e o uso de polimorfismo na prática:

```
using System;

class Character {
    public string Name { get; set; }
    public Character(string name) {
        Name = name;
    }
    public virtual void Attack() {
        Console.WriteLine($"{Name} ataca de forma genérica.");
    }
}

class Warrior : Character {
    public Warrior(string name) : base(name) { }
    public override void Attack() {
        Console.WriteLine($"{Name} avança com sua espada, desferindo um golpe poderoso!");
    }
}

class Mage : Character {
    public Mage(string name) : base(name) { }
    public override void Attack() {
        Console.WriteLine($"{Name} lança uma bola de fogo ardente contra o inimigo!");
    }
}

// Código de demonstração
class Program {
    static void Main() {
        Character[] party = new Character[2];
        party[0] = new Warrior("Arus");
        party[1] = new Mage("Luna");
        foreach (Character member in party) {
            member.Attack();
        }
    }
}
```


Nesse exemplo, `Character` declara o método `Attack()` como `virtual`, permitindo que subclasses o sobrescrevam. `Warrior` e `Mage` herdam de `Character` e cada um implementa sua versão própria de `Attack()`. No método `Main()`, criamos um array de `Character` contendo instâncias de `Warrior` e `Mage`. Ao iterar por esse array, a chamada `member.Attack()` – na qual `member` é do tipo da classe base `Character` – será resolvida em tempo de execução para invocar a implementação específica daquele objeto: se o objeto for do tipo `Warrior`, executará o código sobrescrito em `Warrior.Attack`; se for do tipo `Mage`, executará `Mage.Attack`. Assim, mesmo usando a referência do tipo genérico `Character`, o programa consegue despachar dinamicamente a chamada para o método apropriado conforme o tipo concreto do objeto. A saída esperada do programa ilustrará isso claramente. Considerando os objetos com nomes “`Arus`” (guerreiro) e “`Luna`” (maga), o console exibiria, por exemplo:

```
Arus avança com sua espada, desferindo um golpe poderoso!  
Luna lança uma bola de fogo ardente contra o inimigo!
```

Note que cada linha de saída corresponde à implementação definida na respectiva subclasse, demonstrando o polimorfismo em ação. Esse mecanismo interno de despacho virtual é viabilizado pelo runtime do .NET através de tabelas de métodos virtuais (v-table): em tempo de execução, cada objeto sabe quais métodos virtuais concretos ele possui, de acordo com sua classe real, permitindo resolver a chamada para o código correto. Do ponto de vista do desenvolvedor, basta marcar os métodos com `virtual/override` e usar referências da superclasse; o runtime se encarrega de invocar a versão adequada.

É importante destacar que, embora a decisão de qual implementação executar seja adiada para o tempo de execução, a resolução de quais métodos existem e podem ser chamados (com base no tipo estático da referência) ocorre em tempo de compilação.

O C# é uma linguagem estaticamente tipada: o compilador verifica os membros disponíveis a partir do tipo declarado da variável. No nosso exemplo, sabemos que `member` é um `Character` e possui um método `Attack()`. O que o compilador não sabe – e nem precisa saber – é qual classe derivada específica `member` referencia em um dado momento. Essa escolha, como vimos, fica para o runtime decidir.

Conforme Skeet (2019) observa, a escolha de qual implementação de um método virtual deve ser executada depende do tipo em tempo de execução do objeto; já o processo de associação da chamada ao nome e à assinatura do método acontece todo em tempo de compilação.

Em outras palavras, o código é compilado esperando invocar algum `Attack()` válido em `Character`, mas só em execução é determinado se essa invocação pula para `Warrior.Attack` ou `Mage.Attack` (ou para a versão base, caso não houvesse `override`). Esse comportamento exemplifica a essência do polimorfismo de subtipo em linguagens OO: o objeto sabe se autosselecionar para executar o método apropriado à sua classe real.

Agora, além do polimorfismo baseado em herança de classes, o C# fornece também o polimorfismo por interface. Vimos que interfaces em C# são tipos especiais que definem um conjunto de membros (métodos, propriedades, indexadores, eventos etc.) que representam uma capacidade ou um comportamento, porém sem implementar nenhum deles – isto é, uma interface contém apenas as assinaturas dos membros, cabendo às classes ou às estruturas que a implementam fornecer o código desses membros.

Em termos conceituais, uma interface define um contrato ao qual uma classe (ou struct) pode aderir. Quando uma classe implementa uma interface, ela se compromete a cumprir aquele contrato, ou seja, a prover implementações concretas para todos os membros declarados na interface. Sintaticamente, declara-se uma interface em C# usando a palavra-chave **interface**. Diferentemente de uma classe, uma interface não estende nenhuma classe base (nem mesmo `System.Object` é explicitamente colocado, ainda que, internamente, todos os tipos em .NET derivem de `Object`).

Além disso, todos os membros de uma interface são implicitamente públicos e abstratos, por isso, na definição da interface, não se coloca modificador de acesso nem corpo nos métodos/propriedades. Qualquer tentativa de incluir implementação dentro de uma interface tradicional resultaria em erro de compilação. Historicamente, interfaces em C# serviam estritamente como contratos puros, definindo o que deve ser feito, mas nunca como.



Observação

A partir do C# 8, foi introduzido o conceito de métodos de interface `default` ou defendidos, que podem conter uma implementação padrão dentro da interface. No entanto, essa funcionalidade avançada tem como objetivo principal permitir evoluir interfaces sem quebrar código legado; ela não muda a natureza fundamental do polimorfismo por interface, pois as classes ainda podem sobrescrever esses métodos `default` se desejarem. Para simplificar, focaremos o caso típico em que a interface não tem implementação.

Quando uma classe ou uma estrutura declara que implementa uma interface (usando o símbolo `:` na definição da classe, seguido do nome da interface), ela deve obrigatoriamente fornecer a implementação de todos os membros definidos nessa interface; caso contrário, a classe deve ser marcada ela própria como `abstract` (incompleta).

Em outras palavras, implementar uma interface é uma operação do tipo tudo ou nada: o tipo que se compromete com uma interface não pode escolher alguns membros para implementar e ignorar outros; ele precisa escrever todos, mesmo que alguns simplesmente deleguem a outras lógicas internas.

Por exemplo, se uma interface declarar dez métodos, qualquer classe concreta que a implemente terá de escrever as dez funções correspondentes (com a assinatura exata especificada pela interface).

Essa característica garante que qualquer instância de uma classe que implemente a interface realmente disponha de todos os membros do contrato, evitando chamadas indefinidas.

Um ponto poderoso das interfaces é que elas permitem uma forma de herança múltipla de comportamento no C#. Uma classe pode estender de outra classe e, ao mesmo tempo, implementar várias interfaces adicionais. De fato, uma classe em C# pode implementar quantas interfaces forem necessárias.

A sintaxe da linguagem suporta isso separando cada interface por vírgula após o nome da classe base (se a classe não herdar de outra classe, ela pode implementar interfaces diretamente após os dois-pontos). Por exemplo:

```
class MinhaClasse : MinhaBase, IPrimeiraInterface, ISegundaInterface { ... }.
```

No caso de interfaces em cascata, uma interface também pode herdar (estender) de outras interfaces, agregando seus contratos (embora interfaces não possam derivar de classes). Essa flexibilidade permite modelar em C# situações em que diferentes tipos precisam cumprir um mesmo conjunto de operações, sem que necessariamente estejam na mesma hierarquia de classes.

Inclusive, esse é um dos critérios de escolha entre usar herança de classes abstratas e usar interfaces: quando se quer aproveitar código comum e há claramente uma relação é-um entre os tipos, pode-se usar uma classe base; mas, quando se quer apenas impor comportamentos comuns a tipos potencialmente não relacionados, interfaces são a escolha ideal.

Além disso, interfaces eliminam a limitação de herança simples das classes, já que um tipo pode herdar (no sentido de implementar) múltiplos contratos de interface.

Para ilustrar o polimorfismo via interfaces, considere um exemplo também inspirado em jogos. Suponha que desejamos modelar objetos no jogo que podem sofrer dano (por exemplo, personagens e elementos do cenário destruíveis). Podemos definir uma interface **IDamageable** (em português, danificável, ou seja, suscetível a dano) que declara um método **TakeDamage(int amount)**, o qual representa a ação de receber um certo montante de dano.

Duas classes distintas – por exemplo, **Player** (Jogador) e **Barrel** (Barril explosivo) – podem implementar essa interface, cada uma à sua maneira. O jogador, ao sofrer dano, reduz suas unidades de vida (**Health**); já o barril, ao sofrer dano, reduz sua durabilidade até eventualmente explodir. Essas classes não precisam ter qualquer relação de herança entre si; basta que ambas implementem **IDamageable**. Podemos então escrever código que trate coleções de objetos **IDamageable** genericamente, aplicando dano a todos, sem se importar se o objeto é jogador ou barril, pois cada um reagirá de forma distinta. Veja o código:

```
using System;

interface IDamageable {
    void TakeDamage(int amount);
}
```

```

class Player : IDamageable {
    public string Name { get; set; }
    public int Health { get; private set; }
    public Player(string name, int health) {
        Name = name;
        Health = health;
    }
    public void TakeDamage(int amount) {
        Health -= amount;
        Console.WriteLine($"{Name} recebeu {amount} de dano. Saúde restante:
{Health}");
        if (Health <= 0) {
            Console.WriteLine($"{Name} foi derrotado!");
        }
    }
}

class Barrel : IDamageable {
    public int Durability { get; private set; }
    public Barrel(int durability) {
        Durability = durability;
    }
    public void TakeDamage(int amount) {
        Durability -= amount;
        Console.WriteLine($"Barril sofreu {amount} de dano. Durabilidade
restante: {Durability}");
        if (Durability <= 0) {
            Console.WriteLine("O barril quebrou em pedaços!");
        }
    }
}

// Código de demonstração
class Program {
    static void Main() {
        IDamageable[] targets = new IDamageable[2];
        targets[0] = new Player("Hero", 100);
        targets[1] = new Barrel(50);
        foreach (IDamageable target in targets) {
            target.TakeDamage(30);
        }
    }
}

```

Nesse exemplo, definimos a interface **IDamageable** com o método **TakeDamage(int)**. As classes **Player** e **Barrel** implementam essa interface, cada uma fornecendo a lógica apropriada para lidar com dano. Em **Main()**, criamos um array do tipo **IDamageable** e armazenamos um **Player** e um **Barrel** nessa coleção – note que isso é permitido porque ambos os objetos são **IDamageable**, ou seja, satisfazem o contrato da interface.

Ao iterar sobre **targets** e chamar **target.TakeDamage(30)**, o programa está, de forma polimórfica, invocando o método da interface sem saber qual é a classe concreta de **target** em cada iteração. Novamente, graças ao polimorfismo, cada chamada a **TakeDamage** será direcionada, em

tempo de execução, à implementação correspondente à classe real do objeto: na primeira iteração, `target` é um `Player`, então se executa `Player.TakeDamage`; na segunda, `target` é um `Barrel`, então se executa `Barrel.TakeDamage`. O resultado impresso poderia ser:

```
Hero recebeu 30 de dano. Saúde restante: 70
Barril sofreu 30 de dano. Durabilidade restante: 20
```

Cada objeto respondeu de forma distinta ao mesmo estímulo (`TakeDamage(30)`), conforme sua classe. Se chamássemos o método novamente mais algumas vezes, eventualmente, o barril quebraria (quando sua durabilidade chegasse a zero ou menos) e o jogador seria derrotado (quando sua saúde ficasse igual a zero ou negativa), de acordo com as condições programadas em cada implementação.

Esse exemplo evidencia como o uso de interfaces permite escrever código genérico (no caso, um loop aplicando dano) que lida com objetos de tipos diversos de forma unificada. Ainda, o polimorfismo por interface garante que todos os objetos em questão sabem realizar aquela operação (pois implementam o contrato), embora cada classe a realize de maneira própria.

Internamente, a chamada de um método através de uma interface funciona, no runtime, de modo análogo à chamada de um método virtual de classe: o CLR utiliza tabelas de despacho para interfaces (às vezes chamadas de interface method tables) a fim de descobrir qual função concreta invocar no objeto. Assim, a invocação `target.TakeDamage(30)` será resolvida para o endereço do método `TakeDamage` implementado pela classe real de `target` (seja `Player` ou `Barrel`).

O despacho é decidido dinamicamente com base no tipo em tempo de execução do objeto. Do ponto de vista do código, `target` é conhecido apenas como `IDamageable` (o compilador garante que esse tipo declara o método `TakeDamage(int)`), mas, no momento da execução do método, o runtime verifica a verdadeira classe de `target` e encaminha a chamada para a implementação correta daquela classe.

Esse mecanismo permite um **acoplamento muito fraco** entre quem chama e quem executa a ação. Assim, o código chamador não precisa saber nada sobre como o objeto irá cumprir a tarefa de receber dano, apenas confia que ele a cumprirá porque implementa a interface esperada.



Saiba mais

O livro a seguir é uma obra clássica de design orientado a objetos que introduz os padrões GRASP, entre eles baixo acoplamento e polimorfismo, discutindo detalhadamente como estruturar classes e interfaces para minimizar dependências e favorecer extensibilidade.

LARMAN, C. *Utilizando UML e padrões: uma introdução à análise e ao projeto orientados a objetos e ao desenvolvimento iterativo*. 3. ed. Porto Alegre: Bookman, 2007.

Tanto o polimorfismo via classes virtuais quanto o polimorfismo via interfaces apresentam vantagens e papéis complementares no design de software em C#. Uma dúvida comum é: quando usar herança (com métodos virtuais) e quando usar interfaces?

A resposta depende do relacionamento conceitual entre os tipos e do reaproveitamento de código desejado. De modo geral, a herança de classes é adequada quando existe uma relação clara de generalização/especialização (como uma relação é-um, onde um **Mage** é um **Character**) e quando faz sentido prover alguma implementação padrão ou estado compartilhado na superclasse. Com a herança, podemos centralizar código comum na classe base e sobrescrever apenas diferenças nas subclasses, além de podermos armazenar estado (atributos) que todas as subclasses herdam.

Por outro lado, as interfaces são indicadas quando queremos definir um conjunto de operações que podem ser realizadas por objetos de classes completamente distintas, potencialmente sem nenhuma relação de herança entre si, ou mesmo pertencentes a hierarquias diferentes. Interfaces promovem um menor acoplamento – o código consumidor depende apenas da interface, e não de uma classe específica – e permitem a um tipo adquirir múltiplas capacidades sem quebrar o modelo de herança simples da linguagem.

Por exemplo, poderíamos ter uma classe **Mage** que herda de **Character** e também implementa interfaces como **IDamageable** (se magos também pudessem sofrer dano) e talvez uma interface **ICastSpell** (indicando que o mago pode lançar magias). Isso seria difícil de modelar apenas com herança, já que C# não possibilita herança múltipla de classes. Assim, interfaces suprem essa necessidade ao permitir que uma classe realize diversos contratos diferentes.

Em termos de desempenho, chamadas polimórficas (seja via métodos virtuais ou interfaces) introduzem um pequeno overhead (custo extra de processamento) em relação a chamadas diretas (estáticas), pois exigem uma indireção para resolver o método em tempo de execução. No entanto, no ambiente .NET moderno, esse overhead é geralmente muito baixo, dada a otimização do JIT compiler e do runtime.

Em cenários usuais de alto nível, a clareza e a extensibilidade fornecidas pelo polimorfismo superam em muito qualquer custo de performance. Apenas em casos de exigência extrema de desempenho é que pode ser necessário considerar alternativas (como devirtualization em cenários específicos, ou evitar interfaces em loops críticos), mas essas são situações excepcionais. Resumindo, o despacho virtual em C# permite que métodos sobrepostos em classes derivadas sejam chamados corretamente conforme o tipo real do objeto, viabilizando o polimorfismo clássico de herança. Já o polimorfismo por interface possibilita que objetos de classes diferentes, sem relação direta de herança, sejam tratados uniformemente se compartilharem um contrato comum.

Ambos os mecanismos reforçam o LSP – ou seja, um objeto de uma subclasse ou de uma classe que implementa uma interface pode ser utilizado no lugar de um objeto do tipo base (seja a superclasse ou a própria interface), com a garantia de que as operações invocadas se comportarão de maneira válida para aquele tipo concreto.

Esses recursos são essenciais para escrever código extensível e de fácil manutenção: novas classes podem ser adicionadas ao sistema implementando interfaces existentes ou herdando de classes base polimórficas, e o código cliente imediatamente pode utilizá-las sem precisar ser modificado.



Resumo

Esta unidade explorou as formas clássicas de modelar entidades, comportamentos e relacionamentos, tomando como eixo as decisões de design em torno de herança, classes abstratas e seladas, composição de objetos e polimorfismo, sempre com o apoio de pequenos trechos de código ilustrativo.

Num primeiro momento, concentramo-nos nas hierarquias de herança e no papel das classes abstratas e seladas. Discutimos as relações é-um entre tipos, o reúso white-box proporcionado pela herança e seus riscos, como o fragile base class problem e a rigidez de grandes árvores de herança, ponto em que surge a restrição da herança simples em C#.

Nesse contexto, aparece a utilidade de classes abstratas como contratos parciais para especializações futuras, bem como o uso da palavra-chave sealed para encerrar intencionalmente uma hierarquia, ilustrado inclusive com referências a tipos selados do .NET.

Em paralelo, a composição é apresentada como alternativa baseada em relações tem-um, com destaque para o padrão entidade-componente em jogos e para o princípio clássico "prefira composição em vez de herança", exemplificado pelo redesenho do cenário de personagens e armas.

Na sequência, o foco recaiu sobre o comportamento dinâmico dos objetos, com a análise do despacho virtual e do polimorfismo obtido por meio de métodos virtuais e abstratos. Detalhamos o uso de virtual e override em hierarquias de classes, esclarecendo as regras da linguagem, a distinção entre métodos virtuais e abstratos e o papel das tabelas de métodos virtuais no runtime do .NET.

Um cenário didático com a classe Character e suas especializações Warrior e Mage ilustrou como a resolução de chamadas em tempo de execução permite tratar coleções de personagens de forma uniforme, preservando a individualidade de cada implementação de Attack e aproximando a discussão de princípios como o LSP.

Por fim, ampliamos a discussão ao polimorfismo por interface, mostrando como interfaces em C# definem contratos que podem ser implementados por tipos de hierarquias distintas, contornando a limitação da herança simples e reduzindo o acoplamento. O exemplo de IDamageable implementada tanto por um Player quanto por um Barrel evidenciou como o código

cliente pode operar sobre um conjunto heterogêneo de objetos com base em uma capacidade comum, sem conhecer seus detalhes concretos.

Amarrando esses tópicos, esta unidade contrasta critérios práticos de uso de herança, composição e interfaces, dialogando com a literatura de design orientado a objetos (padrões de projeto, GRASP, SOLID) e preparando o leitor para avaliar, em cenários reais de desenvolvimento de jogos e de outras aplicações, quais mecanismos de C# melhor sustentam reúso, extensibilidade e baixo acoplamento.



Exercícios

Questão 1. Uma classe `CumprimentoMulti` precisa oferecer saudações em inglês e em francês por meio de dois contratos distintos, `IGreetingsEn` e `IGreetingsFr`, ambos com o método `Saudar()`. O requisito é que a saudação em inglês permaneça acessível diretamente na API pública da classe, enquanto a saudação em francês só deve ser acessível quando o objeto for tratado pelo tipo da interface francesa. Considerando contratos por interfaces e a implementação explícita em C#, qual alternativa é a mais adequada?

- A) Implementar apenas um método público `Saudar()` e devolver as duas saudações concatenadas, pois isso atende aos dois contratos com uma única implementação.
- B) Criar dois métodos públicos com o mesmo nome `Saudar()` e com a mesma assinatura, pois o compilador escolherá o correto em tempo de execução com base na interface usada.
- C) Implementar `IGreetingsEn.Saudar()` de modo implícito (método público) e implementar `IGreetingsFr.Saudar()` de modo explícito (qualificando o nome com a interface), exigindo o uso da referência `IGreetingsFr` para acessar a versão em francês.
- D) Implementar os dois métodos como públicos e, dentro deles, lançar exceção quando o chamador não estiver no modo correto, pois isso preserva o contrato e força o uso adequado.
- E) Evitar interfaces e usar uma única classe base abstrata `Greetings`, pois isso permite que a mesma classe herde de duas bases e elimine conflitos de nomes.

Resposta correta: alternativa C.

Análise das alternativas

A) Alternativa incorreta.

Justificativa: concatenar respostas altera a semântica do contrato: cada interface espera uma saudação específica, e não uma combinação arbitrária.

B) Alternativa incorreta.

Justificativa: não é permitido ter dois métodos com a mesma assinatura no mesmo tipo, e a resolução não ocorre por escolha dinâmica nesse caso. É preciso separar as implementações pelo mecanismo apropriado.

C) Alternativa correta.

Justificativa: a implementação explícita permite cumprir dois contratos homônimos sem expor ambos na interface pública usual da classe: a versão explícita só aparece quando o objeto é visto pela interface correspondente, o que atende ao requisito de visibilidade.

D) Alternativa incorreta.

Justificativa: lançar exceções por contexto de chamada tende a criar um contrato frágil para o código cliente. O objetivo é oferecer implementações coerentes por contrato, e não aceitar a chamada nem falhar por uma regra externa ao tipo.

E) Alternativa incorreta.

Justificativa: em C#, uma classe não pode herdar de duas classes base. Além disso, o problema descrito é de contratos com interfaces homônimas, resolvido de forma direta com a implementação explícita.

Questão 2. Em um jogo em C#, o time criou uma hierarquia `Personagem` -> `Guerreiro`/`Arqueiro` para reutilizar código e habilitar polimorfismo. Com o tempo, a classe base `Personagem` passou a concentrar várias regras, e mudanças nela começaram a quebrar subclasses existentes, exigindo correções em cadeia. Considerando o problema da classe base frágil e a distinção entre reuso caixa-branca (herança) e reuso caixa-preta (composição), qual ação é a mais coerente para reduzir a fragilidade e manter a flexibilidade?

- A) Aumentar a profundidade da hierarquia, criando mais níveis intermediários (por exemplo, `Personagem` -> `PersonagemHumano` -> `Guerreiro`), pois hierarquias profundas isolam mudanças e evitam impactos colaterais.
- B) Consolidar todos os comportamentos variantes na classe base `Personagem` e impedir `override`, pois centralizar o comportamento elimina divergências entre subclasses.
- C) Substituir parte da variação de comportamento por composição, delegando capacidades variáveis (por exemplo, o modo de ataque) a componentes (por exemplo, `Arma`) e mantendo a herança apenas quando houver relação conceitual clara do tipo é-um.
- D) Forçar que toda subclasse seja `sealed`, pois classes seladas evitam bugs e tornam o sistema automaticamente mais extensível.
- E) Evitar a composição e usar somente interfaces, pois interfaces sempre substituem herança e dispensam qualquer implementação compartilhada.

Resposta correta: alternativa C.

Análise das alternativas

A) Alternativa incorreta.

Justificativa: hierarquias mais profundas tendem a ampliar o acoplamento estrutural e a área de impacto de mudanças, o que costuma agravar o risco de fragilidade em vez de mitigá-lo.

B) Alternativa incorreta.

Justificativa: impedir sobrescrita elimina um dos principais ganhos de polimorfismo e força um comportamento único, o que reduz a capacidade de especialização e não ataca a causa do problema (o acoplamento da subclasse com detalhes da base).

C) Alternativa correta.

Justificativa: a composição trata componentes como caixas-pretas, reduzindo a dependência de detalhes internos e permitindo trocar o comportamento em tempo de execução (por exemplo, trocar `Espada` por `Arco`). Isso diminui a fragilidade associada ao reuso caixa-branca da herança e evita a explosão combinatória de subclasses.

D) Alternativa incorreta.

Justificativa: selar subclasses reduz extensões futuras e pode ser útil pontualmente, mas não resolve o problema de acoplamento entre base e derivadas. Além disso, a extensibilidade não decorre de impedir herança indiscriminadamente.

E) Alternativa incorreta.

Justificativa: interfaces definem contratos, mas não fornecem, por si, uma estratégia de reutilização de implementação e de estado compartilhado. Em muitos cenários, composição e interfaces atuam juntas, e não como substitutas universais.
